Beacon

Dissecting a malicious macro document: VBA analysis, shellcode extraction, payload decoding, and C2 configuration

Scenario	A suspicious macro-enabled spreadsheet was recovered during an investigation. Analysis is required to inspect the underlying VBA code, extract embedded shellcode, and uncover command-and-control infrastructure.
Objective	Analyze the file, deobfuscate embedded macros, extract raw shellcode, identify the payload family, and map C2 configuration settings.
Sample SHA-256	`01dac009a652c9f592a6ea6003daaa2e198b4761240a89af5b5bf80d737e2e25`
Verification Link	https://blueteamlabs.online/achievement/share/61222/174
Tools	Linux Terminal, sha256sum, oledump.py, Python scripts, CyberChef, VirusTotal
Approach	Hash verification, OLE stream extraction, VBA code analysis, shellcode extraction and Base64 decoding, memory API analysis, and C2 profile parsing.

SUMMARY

The analyzed sample is an Excel document named `invoice.xlsm` carrying an obfuscated VBA macro payload. Upon inspection, the document utilizes Win32 API functions including `VirtualAlloc`, `RtlMoveMemory`, and `CreateThread` to allocate executable memory space directly within the host process and execute decoded shellcode. Deobfuscation and Base64 extraction yields a secondary payload matching a Cobalt Strike Beacon staged loader. Parsing the payload configuration reveals an active HTTPS Command-and-Control channel communicating with `aicsoftware.com` over port 757, tied directly to a specific Cobalt Strike 4.2 framework release.

PHASE 1 SAMPLE IDENTIFICATION & OLE STREAM INSPECTION

The investigation began by calculating the cryptographic hash of `invoice.xlsm` within the analysis environment using `sha256sum`. Computing the SHA-256 hash establishes a clean tracking identifier for the sample prior to execution or extraction.

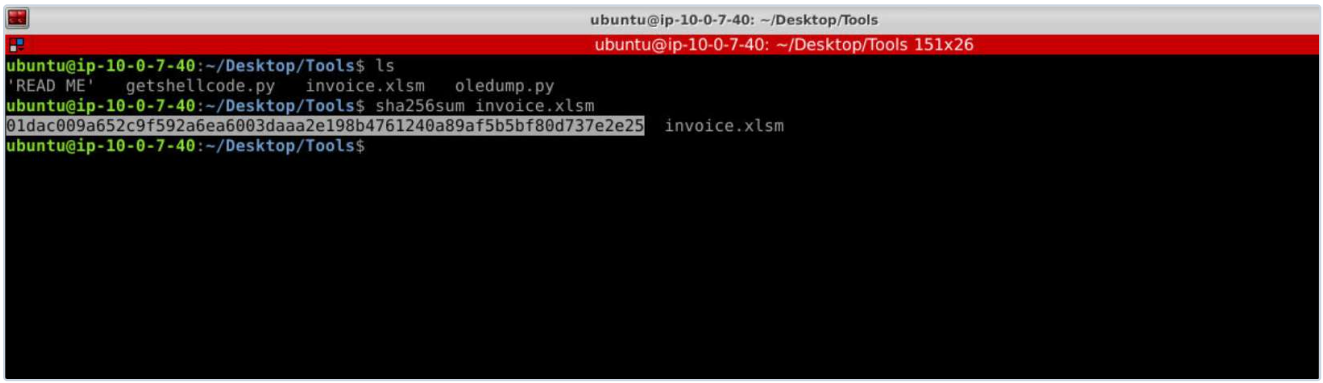


Figure 1 - Initial SHA-256 computation of invoice.xlsm.

Next, `oledump.py` was executed against `invoice.xlsm` to inspect the internal OLE stream hierarchy of the macro container. Streams marked with an indicator of **'M'** contain executable VBA code. The analysis identified three key macro streams: stream 3 (`VBA/Payables`), stream 4 (`VBA/Receivables`), and stream 6 (`VBA/ThisWorkbook`).

```
ubuntu@ip-10-0-7-40:~/Desktop/Tools$ python oledump.py invoice.xlsm
A: xl/vbaProject.bin
 A1:       537 'PROJECT'
 A2:       125 'PROJECTwm'
 A3: M    3903 'VBA/Payables'
 A4: M  825225 'VBA/Receivables'
 A5: m     999 'VBA/Sheet1'
 A6: M    4222 'VBA/ThisWorkbook'
 A7:      3969 'VBA/_VBA_PROJECT'
 A8:    772193 'VBA/__SRP_0'
 A9:       559 'VBA/__SRP_1'
A10:      2792 'VBA/__SRP_2'
A11:      6067 'VBA/__SRP_3'
A12:       269 'VBA/__SRP_4'
A13:       156 'VBA/__SRP_5'
A14:       156 'VBA/__SRP_6'
A15:       784 'VBA/__SRP_7'
A16:       617 'VBA/dir'
ubuntu@ip-10-0-7-40:~/Desktop/Tools$
```

Figure 2 - OLE stream structure showing streams A3, A4, and A6 containing VBA macros.

PHASE 2 MACRO ANALYSIS AND EXECUTION HIJACKING

Extracting stream 6 (`ThisWorkbook`) shows the auto-execution entry point. The macro defines an automated hook `Workbook_Open()` that triggers immediately when macros are enabled. It executes two internal routines, `Mergers` and `Acquisitions`, masked among obfuscated junk text referencing financial ledgers.

```
ubuntu@ip-10-0-7-40:~/Desktop/Tools$ cat s6
Attribute VB_Name = "ThisWorkbook"
Attribute VB_Base = "0{00020819-0000-0000-C000-000000000046}"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = True
Attribute VB_TemplateDerived = False
Attribute VB_Customizable = True
Public Sub Workbook_Open()
'Within the dimly lit caverns of Spreadsheet Sanctum, there resides the Matrix of Muddled Invoicing.
'This isn't your everyday spreadsheet; it's a matrix dedicated to the nebulous notations procured from the Consortium of Confounded Commerce.
'Every cell within this matrix is judiciously jotted on Tab Tau-29, a digital datasheet that legend suggests was initiated by the inaugural initiates o
f indirect invoicing.
'The chief column, denominated Indeterminate Invoice Inputs, enlists amounts that might be either billings or baffling blips.
'These values aren't anchored in actual trade but are postulations posited by the enigmatic entity known as Guesswork Galore Group,
'a shadowy syndicate steeped in spreadsheet speculation.
'Monthly maneuverings in the Matrix of Muddled Invoicing are less about logic and more about the labyrinthine lore of latent ledgers.
'Here, figures don't necessarily flow; they float, fostering feelings of fiscal fantasy.
'One might ponder: In a spreadsheet shrouded in such shadows, can clarity ever truly be computed?
'As analysts alight upon the aisles of Tab Tau-29, it becomes abundantly apparent that the alleys of ambiguous arithmetic are a maze most mesmerizing.
'Here, precision pivots towards perplexity, and definitive data delves into the domain of the doubtful.
    Mergers
    Acquistions
End Sub
ubuntu@ip-10-0-7-40:~/Desktop/Tools$
```

Figure 3 - Workbook_Open routine triggering macro execution upon opening.

Inspecting stream 4 (`Receivables`) reveals low-level Windows API imports declared using `PtrSafe` for 64-bit architecture compatibility:

- `VirtualAlloc` : Allocates executable memory space initialized with `PAGE_EXECUTE_READWRITE` (`&H40`) permissions.
- `RtlMoveMemory` : Copies decoded byte streams directly into allocated virtual memory space.
- `CreateThread` : Spawns a dedicated execution thread pointing directly to the allocated memory address.

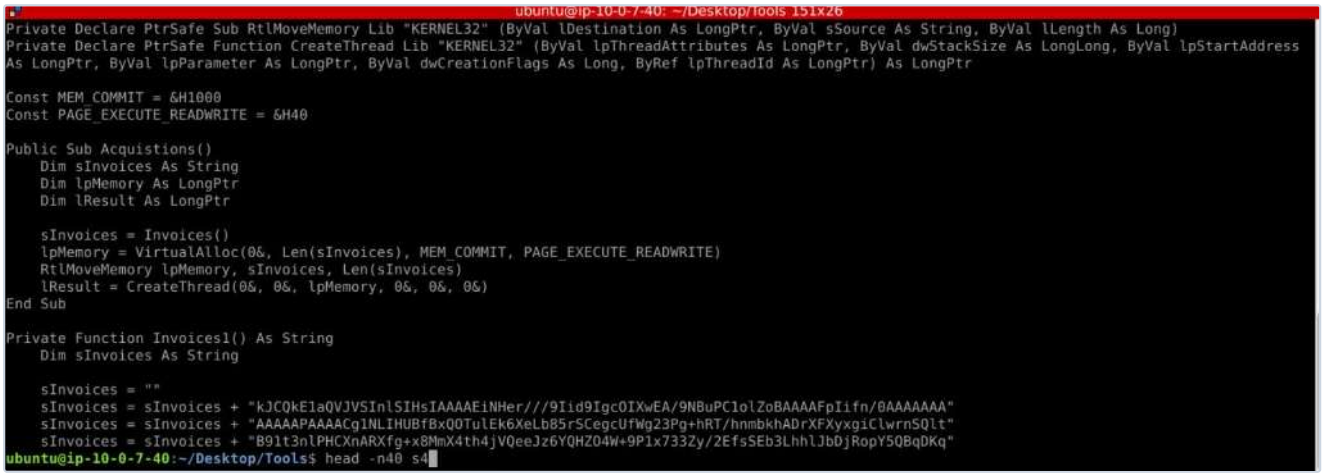


Figure 4 - Win32 API declarations for VirtualAlloc, RtlMoveMemory, and CreateThread.

Reviewing stream 4 further shows how the payload data is fetched from the `Invoices()` function and prepared for thread creation. Stream 3 (`Payables`) contains string fragments concatenated into a single large Base64 buffer named `sInvoices`.

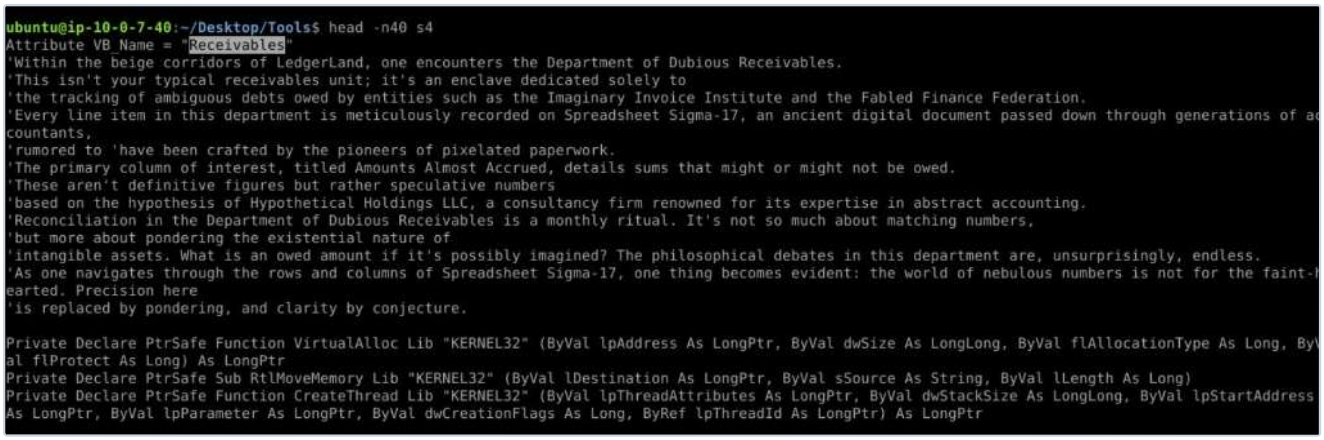


Figure 5 - Receivables module declaring subroutines for payload injection.

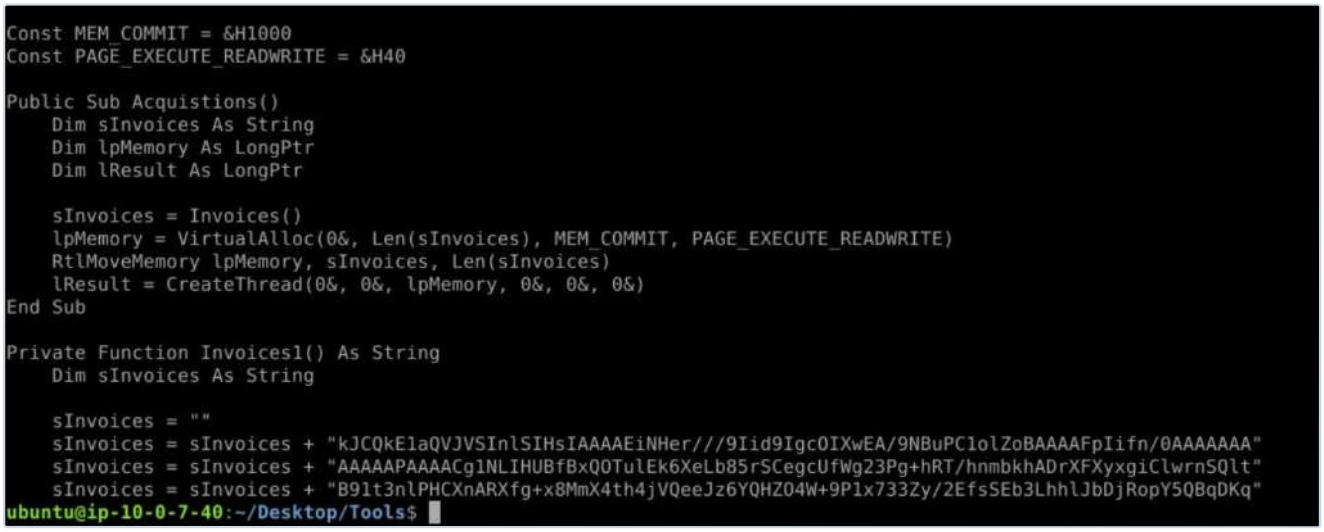


Figure 6 - Concatenation of encoded payload string sInvoices within stream 3.

PHASE 3 SHELLCODE EXTRACTION AND PAYLOAD IDENTIFICATION

To analyze the payload without executing the macro, a Python script (`getshellcode.py`) was executed against stream 4 (`s4`) to carve out the full string and write it into `extracted_payload`. Piping this raw stream through `base64 -d` generated the decoded binary artifact named `payload`.

Calculating the SHA-256 hash of `payload` returned `fe143d2a4e74094c076bd72bd144ee1cfb4764bb62545a252113bff470011123`.

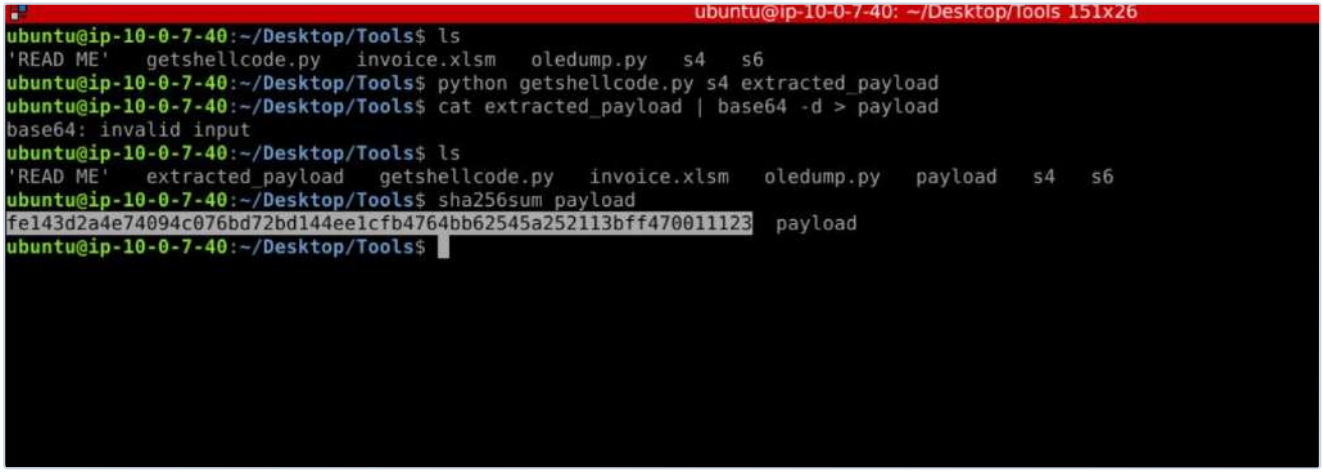


Figure 7 - Shellcode extraction, Base64 decoding, and binary hash computation.

Querying the payload hash across threat intelligence databases matched multiple YARA signatures for Cobalt Strike Beacon loaders, specifically identifying reflective DLL launcher techniques and dynamic sandbox detections for `trojan.cobaltstrike/beacon`.

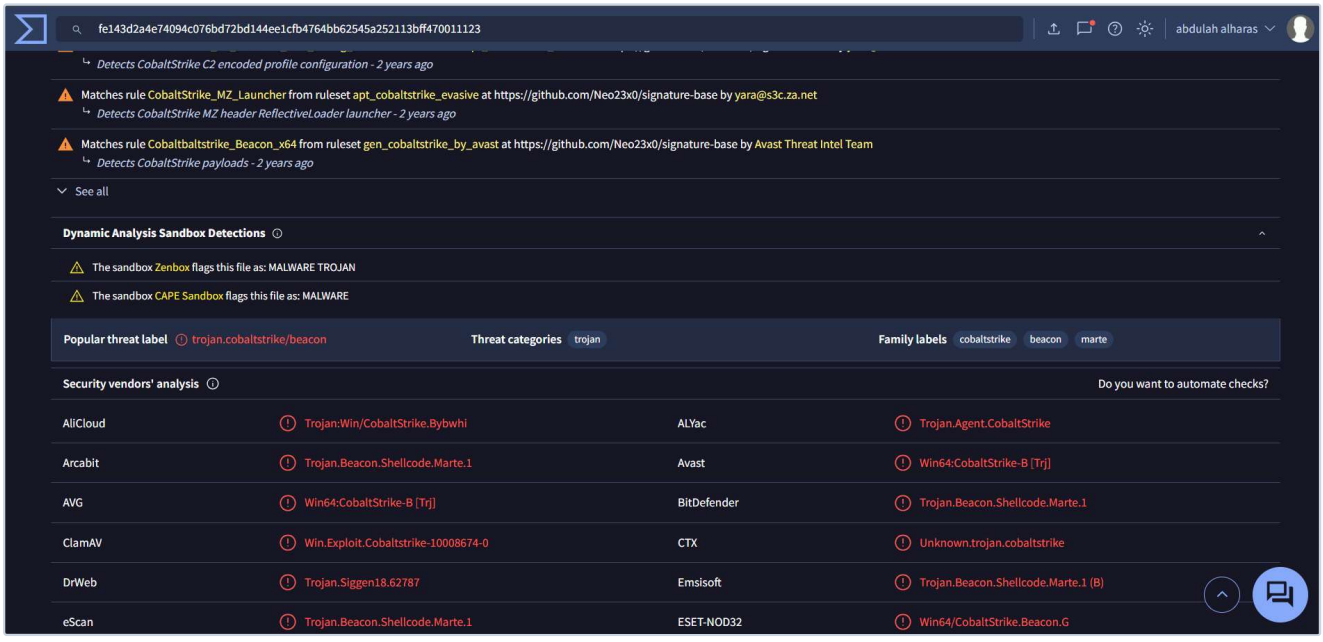


Figure 8 - Threat intelligence detection matching Cobalt Strike Beacon signatures.

PHASE 4 COMMAND-AND-CONTROL (C2) PROFILE PARSING

Running the parsed shellcode through a Beacon configuration extractor (`csce`) dumped the internal C2 profile settings into structured JSON data. Key operational parameters extracted include:

- **Beacon Type:** HTTPS
- **C2 Hostname:** `aicsoftware.com`
- **C2 Port:** 757
- **HTTP GET URI:** `/templates`
- **HTTP POST URI:** `/favicon`
- **Injection Target:** `%windir%\sysnative\regsvr32.exe`



Figure 9 - Parsed Cobalt Strike Beacon configuration showing host, port, and URIs.

The configuration dump contained a Base64-encoded stub value (`tUr+Aexqde3zXhpE+L05KQ==`). Inputting this value into CyberChef and executing a `From Base64` to `To Hex` conversion yielded the hex string `b54afe01ec6a75edf35e1a44f8bd3929` .

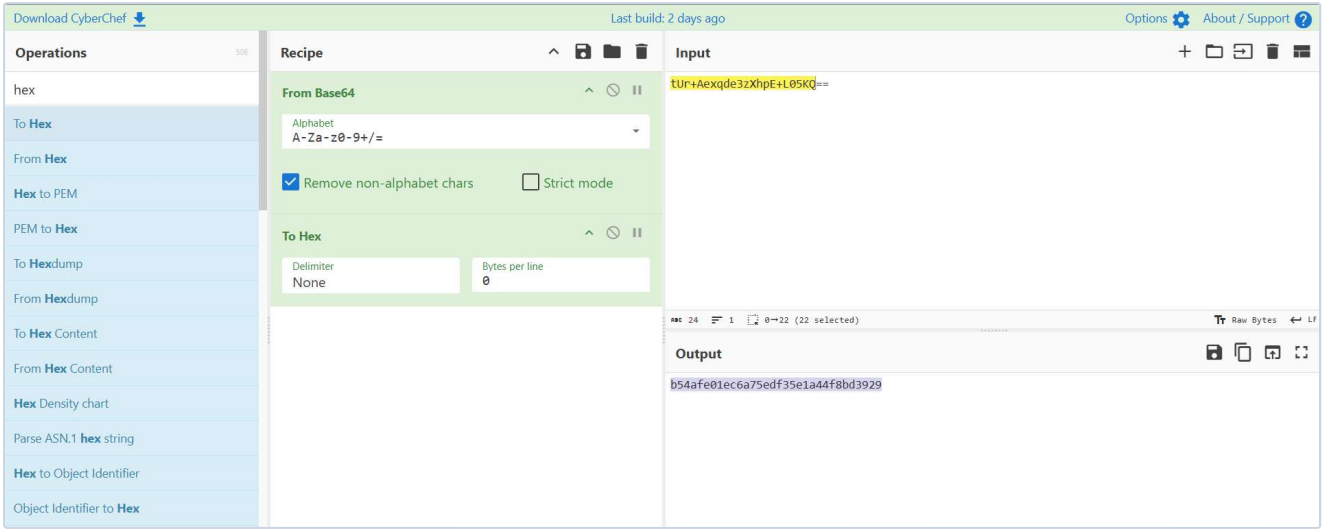


Figure 10 - CyberChef decoding of the public key stub string to hex representation.

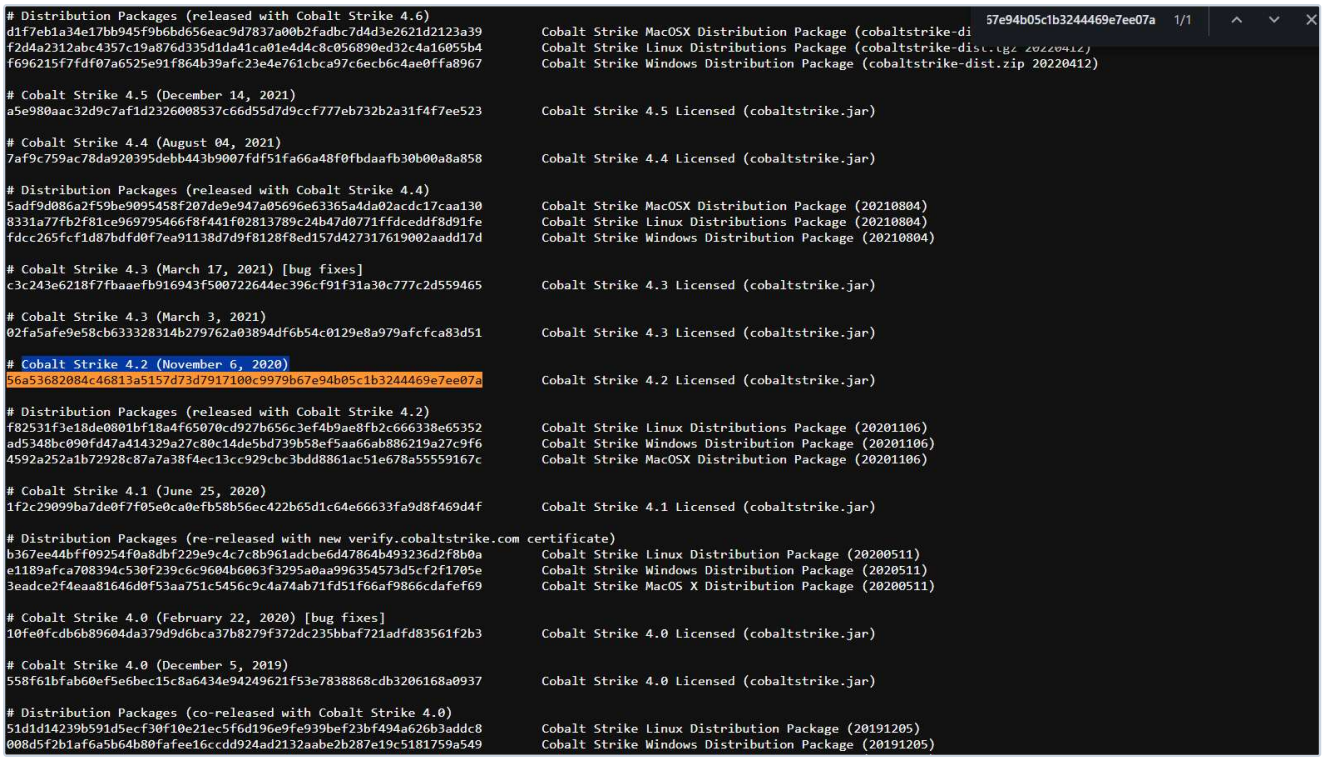
PHASE 5 FRAMEWORK ATTRIBUTION & RELEASE CORRELATION

Analyzing associated binary component hashes from the threat intelligence graph returned the SHA-256 hash `56a53682084c46813a5157d73d7917100c9979b67e94b05c1b3244469e7ee07a` for `cobaltstrike.jar`, flagged by 47 of 62 security vendors.



Figure 11 - Vendor detections for the Cobalt Strike core executable JAR file.

Cross-referencing this hash against known release manifests maps the component directly to **Cobalt Strike 4.2 (Licensed)**, released on November 6, 2020.



```
# Distribution Packages (released with Cobalt Strike 4.6)
d1f7eb1a34e17bb945f9b6bd656eac9d7837a00b2fadbc7d4d3e2621d2123a39        Cobalt Strike MacOSX Distribution Package (cobaltstrike-di
f2d4a2312abc4357c19a876d335d1da41ca01e4d4c8c056890ed32c4a16055b4        Cobalt Strike Linux Distributions Package (cobaltstrike-dist.tgz 20220412)
f696215f7fdf07a6525e91f864b39afc23e4e761cbca97c6ecb6c4ae0ffa8967        Cobalt Strike Windows Distribution Package (cobaltstrike-dist.zip 20220412)

# Cobalt Strike 4.5 (December 14, 2021)
a5e980aac32d9c7af1d2326008537c66d55d7d9ccf777eb732b2a31f4f7ee523        Cobalt Strike 4.5 Licensed (cobaltstrike.jar)

# Cobalt Strike 4.4 (August 04, 2021)
7af9c759ac78da920395debb443b9007fdf51fa66a48f0fbdaafb30b00a8a858        Cobalt Strike 4.4 Licensed (cobaltstrike.jar)

# Distribution Packages (released with Cobalt Strike 4.4)
5adf9d086a2f59be9095458f207de9e947a05696e63365a4da02acdc17caa130        Cobalt Strike MacOSX Distribution Package (20210804)
8331a77fb2f81ce969795466f8f441f02813789c24b47d0771ffdceddf8d91fe        Cobalt Strike Linux Distributions Package (20210804)
fdcc265fcf1d87bdfd0f7ea91138d7d9f8128f8ed157d427317619002aadd17d        Cobalt Strike Windows Distribution Package (20210804)

# Cobalt Strike 4.3 (March 17, 2021) [bug fixes]
c3c243e6218f7fbaaefb916943f500722644ec396cf91f31a30c777c2d559465        Cobalt Strike 4.3 Licensed (cobaltstrike.jar)

# Cobalt Strike 4.3 (March 3, 2021)
02fa5afe9e58cb633328314b279762a03894df6b54c0129e8a979afcfca83d51        Cobalt Strike 4.3 Licensed (cobaltstrike.jar)

# Cobalt Strike 4.2 (November 6, 2020)
56a53682084c46813a5157d73d7917100c9979b67e94b05c1b3244469e7ee07a        Cobalt Strike 4.2 Licensed (cobaltstrike.jar)

# Distribution Packages (released with Cobalt Strike 4.2)
f82531f3e18de0801bf18a4f65070cd927b656c3ef4b9ae8fb2c666338e65352        Cobalt Strike Linux Distributions Package (20201106)
ad5348bc090fd47a414329a27c80c14de5bd739b58ef5aa66ab886219a27c9f6        Cobalt Strike Windows Distribution Package (20201106)
4592a252a1b72928c87a7a38f4ec13cc929cbc3bdd8861ac51e678a55559167c        Cobalt Strike MacOSX Distribution Package (20201106)

# Cobalt Strike 4.1 (June 25, 2020)
1f2c29099ba7de0f7f05e0ca0efb58b56ec422b65d1c64e66633fa9d8f469d4f        Cobalt Strike 4.1 Licensed (cobaltstrike.jar)

# Distribution Packages (re-released with new verify.cobaltstrike.com certificate)
b367ee44bff09254f0a8dbf229e9c4c7c8b961adcbe6d47864b493236d2f8b0a        Cobalt Strike Linux Distribution Package (20200511)
e1189afca708394c530f239c6c9604b6063f3295a0aa996354573d5cf2f1705e        Cobalt Strike Windows Distribution Package (2020511)
3eadce2f4eaa81646d0f53aa751c5456c9c4a74ab71fd51f66af9866cdafef69        Cobalt Strike MacOS X Distribution Package (20200511)

# Cobalt Strike 4.0 (February 22, 2020) [bug fixes]
10fe0fcdb6b89604da379d9d6bca37b8279f372dc235bbaf721adfd83561f2b3        Cobalt Strike 4.0 Licensed (cobaltstrike.jar)

# Cobalt Strike 4.0 (December 5, 2019)
558f61bfab60ef5e6bec15c8a6434e94249621f53e7838868cdb3206168a0937        Cobalt Strike 4.0 Licensed (cobaltstrike.jar)

# Distribution Packages (co-released with Cobalt Strike 4.0)
51d1d14239b591d5ecf30f10e21ec5f6d196e9fe939bef23bf494a626b3addc8        Cobalt Strike Linux Distribution Package (20191205)
008d5f2b1af6a5b64b80fafee16ccdd924ad2132aabe2b287e19c5181759a549        Cobalt Strike Windows Distribution Package (20191205)
```

Figure 12 - Release manifest matching hash 56a53682... to Cobalt Strike 4.2.

KEY FINDINGS

Area	Finding
Initial Stager	Excel macro spreadsheet (`invoice.xlsm`) carrying concatenated Base64 payloads.
Execution Technique	Memory allocation (`VirtualAlloc`) with RWX privileges & remote thread creation.
Payload Family	Cobalt Strike Beacon (Staged Loader / Reflective DLL).
C2 Infrastructure	Domain: `aicsoftware.com` | Port: 757 | Protocol: HTTPS
Target Process Injection	`%windir%\sysnative\regsvr32.exe`

MITRE ATT&CK MAPPING

Tactic	Technique	ID
Initial Access	Spearphishing Attachment	T1566.001
Execution	Command and Scripting Interpreter: Visual Basic	T1059.005
Defense Evasion	Process Injection	T1055
Command and Control	Application Layer Protocol: Web Protocols (HTTPS)	T1071.001

CONCLUSION

Through static OLE extraction and offline string parsing, the malicious document was fully deobfuscated without needing local execution. The analysis traced the full chain from macro stream detection to shellcode carving, revealing an active Cobalt Strike 4.2 HTTPS Beacon configuration targeting sensitive Windows system binaries for injection.